A Circuit-Switched Routing Architecture for Coarse-Grained Reconfigurable Array (CGRA) Cells

Arturo Córdoba Villalobos, Jill Carranza Otárola, Juan Pablo Ureña Madrigal, and Víctor Sánchez Alpízar
Instituto Tecnológico de Costa Rica, Cartago, Costa Rica
E-mail: arturocv16@estudiantec.cr, gcarranza@estudiantec.cr,
juurena@estudiantec.cr and vicsma2409@estudiantec.cr

I. INTRODUCTION

Coarse-Grained Reconfigurable Architectures (CGRAs) have emerged as a promising intermediate solution between fixed-function accelerators and fine-grained FPGAs, offering improved performance–energy trade-offs for compute-intensive applications such as high-performance computing (HPC), signal processing, data analytics, and artificial intelligence (AI). By combining spatial programmability with coarse-grained functional units (FUs), CGRAs enable efficient execution of loop-dominated and dataflow-oriented workloads while maintaining architectural flexibility.

A central component in CGRA design is the interconnection network, which determines how processing elements (PEs) communicate and how effectively data dependencies are mapped onto the hardware. Existing CGRA frameworks typically employ highly configurable switch blocks with extensive connectivity to maximize routing flexibility and mappability. While this approach improves adaptability to diverse dataflow graphs (DFGs), it introduces significant structural complexity, increased logic depth, and higher area overhead, particularly in FPGA implementations.

Recent works have explored trade-offs between compute density and interconnect resources, as well as automated design space exploration (DSE) techniques to optimize metrics such as the Initiation Interval (II) and the II–area product. However, these approaches generally preserve the internal microarchitecture of the switching fabric, focusing primarily on architectural scaling and parameter tuning rather than simplifying the routing mechanism itself. As a result, the interconnect remains a major contributor to resource utilization and scalability limitations.

In this work, we propose a reconfigurable CGRA architecture that directly targets the microarchitecture of the interconnect. Instead of relying on highly flexible routing structures with complex switching and control mechanisms, we design a routing structure based on restricted connectivity under a circuit-switched configuration model. In this approach, routing paths are established during configuration time and remain static during execution, eliminating the need for dynamic arbitration or complex control logic. If the communication needs of the microarchitecture change, the paths can be adapted with the trade-off of requiring a configuration time during which the targeted connections can not be used.

The proposed architecture is implemented and evaluated on an FPGA platform using Xilinx Vivado, enabling the analysis of practical metrics such as LUT utilization and maximum achievable frequency. Through this evaluation, we investigate whether controlled reductions in routing flexibility can lead to meaningful improvements in hardware efficiency for FPGA based CGRA implementations.

II. BACKGROUND AND RELATED WORK

Prior work on CGRA architectures has largely focused on architectural exploration and balancing compute and interconnection resources, typically relying on highly configurable interconnect paradigms based on switch blocks with wide connectivity. Works such as [1]–[3] explore the relationship between compute density and interconnect capacity, showing that switching fabrics often become the primary scalability bottleneck as CGRA arrays grow. Although these approaches introduce modeling frameworks and automated design space exploration techniques to optimize metrics such as the II–area product, they generally retain the same highly flexible routing structures.

Other studies focus on improving the mapping process rather than modifying the architecture

itself. For instance, [4] proposes a runtime hardware-based placement and routing approach for CGRA overlays, exploiting spatial parallelism to accelerate dataflow assignment. While this improves compilation efficiency, it also highlights the complexity introduced by highly configurable routing fabrics.

Recent work suggests that architectural efficiency can be improved by constraining flexibility at different levels of the system. In the compute domain, [5] demonstrates that reducing full configurability and distributing heterogeneous processing elements can reduce area and power overhead while preserving mapping capability. From the communication perspective, [6] shows that simplifying routing structures by eliminating buffers and complex crossbars can reduce hardware cost and improve timing behavior. Similarly, [7] proposes aligning compute and communication provisioning through structured dataflow patterns to reduce reliance on highly flexible routing resources.

Collectively, these studies indicate that excessive flexibility in both compute and interconnect structures often leads to area and power penalties, motivating the exploration of more structured and connectivity-aware interconnect designs.

A. Strengths and Limitations of Existing Approaches

Existing CGRA frameworks provide strong support for architectural exploration and flexible mapping of diverse dataflow graphs. Design space exploration techniques enable systematic evaluation of multiple configurations, while mapping-oriented approaches improve compilation efficiency through faster placement and routing.

However, most architectures rely on highly flexible interconnect fabrics based on crossbar-like switch blocks, which introduce a large number of multiplexers and control elements. As CGRA arrays scale, these switching structures become a major contributor to hardware cost and critical path delay.

Such architectures are well-suited for scenarios requiring high programmability and support for irregular communication patterns. However, they become less efficient in resource-constrained implementations such as FPGA-based CGRAs, where the high flexibility of the routing fabric can lead to unnecessary hardware overhead.

B. Challenges Identified from the State of the Art

Based on the analysis of existing CGRA architectures, several key challenges can be identified:

- **High hardware cost of flexible interconnects.**
  Highly flexible routing fabrics, typically implemented with crossbar-based switch blocks, require a large number of multiplexers and control signals, which significantly increases hardware cost.

- **Overprovisioned routing flexibility.**
  In many CGRA architectures, the routing flexibility provided by the interconnect exceeds the communication requirements of typical dataflow workloads.

- **Interconnect scalability limitations.**
  As CGRA arrays grow, the switching fabric becomes a major contributor to both resource utilization and critical path delay.

- **High FPGA resource consumption.**
  In FPGA implementations, complex routing structures directly translate into increased usage of LUTs, flip-flops, and routing resources.

- **Limited exploration of interconnect microarchitecture.**
  Most prior work focuses on architectural exploration, mapping strategies, or heterogeneous processing elements, while the internal organization of the routing fabric remains largely unchanged.

III. PROPOSED DESIGN

The proposed architecture follows a circuit-switched communication model in which routing paths are configured during a dedicated configuration phase, prior to data transmission. Once established, these communication paths remain static during execution, enabling deterministic data movement between processing elements (PEs) without intermediate buffering, runtime arbitration, or complex flow-control mechanisms. This simplifies the communication infrastructure and reduces interconnect control overhead.

The design adopts a scalable grid-based topology composed of processing elements interconnected through switch modules implemented as crossbars. Figure 1 illustrates the proposed design, where each PE is associated with a local switch.

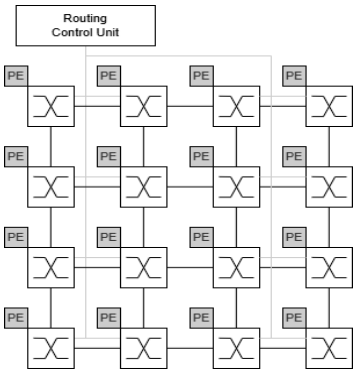

Figure 1. Proposed solution

An alternative switch implementation based on multiplexers was considered. Although multiplexers offer lower area and simpler hardware, they restrict each switch to a single routing direction. In contrast, crossbars require higher area and complexity but provide greater path diversity, enabling multiple simultaneous non-conflicting routes. Specifically, they allow one or more concurrent communication paths depending on source constraints, improving throughput and reducing reconfiguration latency. The proposed architecture maintains efficiency by eliminating buffering and complex flow-control logic.

The Routing Control Unit (RCU) configures the crossbar selection signals during the configuration phase, establishing a static communication fabric that remains fixed during execution without requiring runtime arbitration.

The design can implement multiple routing algorithms, as the way that connections are built across the network can be determined by a higher protocol. For example, optimization algorithms could be applied to redistribute routes optimally for a given design. As this is out of the scope for this work, the algorithm used follows a Manhattan routing scheme over the grid, where data is transferred through sequences of orthogonal hops across adjacent nodes (North, South, East, and West). Each crossbar performs local forwarding according to the preconfigured routing direction, forming deterministic end-to-end paths that correspond to shortest routes in terms of Manhattan distance. Figure 2 illustrates this routing behavior.

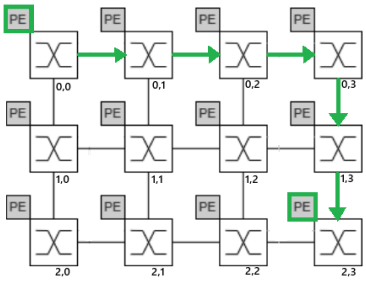

Figure 2. Crossbar flow based on Manhattan routing scheme.

As shown in Figure 2, only the PEs involved in the selected routing path actively participate in the data transfer, either as data sources or destinations, as highlighted in green. The remaining PEs operate in bypass mode, allowing data to traverse the network transparently without modification. In some cases, intermediate PEs may be activated to perform computations or to observe data along the path, depending on the application requirements.

Within each node, the processing element operates independently from the routing logic. When idle, the PE operates in bypass mode, allowing data to traverse the node transparently. When active, it processes incoming data and reinjects the result into the preconfigured routing path. This separation between computation and routing enables continuous pipeline-like data propagation while preserving predictable communication behavior.

During execution, data moves hop-by-hop across the mesh following orthogonal transitions that collectively form a shortest-path route in terms of Manhattan distance. At each intermediate node, the crossbar directs the incoming data toward the next selected direction in the configured path.

There could be cases when a module requires more than one input, for example, a multiplier that requires 2 operands, additional connections are added to the crossbar in such a way that each input can come from a different path. This way, conflicting paths are removed and the arbitration logic becomes simpler. This also sets a max of 3 inputs per node (2 for edge nodes), but in scenarios where more than that is required, a multiplexation in time can be supported to use the same wires for different inputs.

To enable experimental validation of the proposed architecture, a hardware integration layer was implemented to interface the CGRA with the processing system of the Kria KV260 platform. This integration is achieved through a

wrapper module that exposes the configuration and data interfaces of the CGRA through memory-mapped AXI GPIO modules. Through this interface, software running on the processing system can configure routing paths, control data injection, and observe processing element outputs by writing and reading from the corresponding memory-mapped registers. Figure 3 shows the resulting block design used for the hardware implementation and validation of the proposed circuit-switched CGRA architecture.

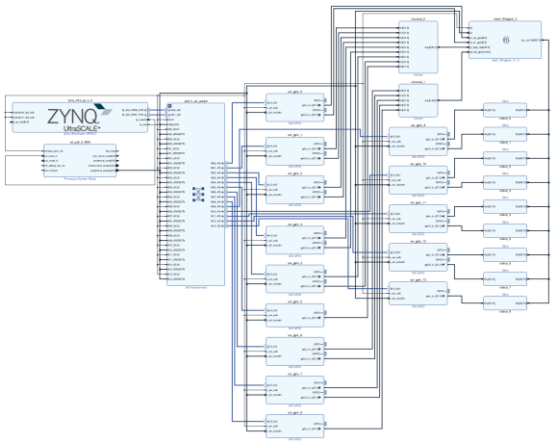

Figure 3. Block design of the 3×3 circuit-switched CGRA interconnect architecture.

The supported connection scenarios are: one-to-one and one-to-many. A scenario of many-to-one could be supported by using the proposed approach mentioned earlier, where the crossbar has additional input/outputs to support the additional inputs from a PE. A scenario of many-to-many is not supported by the proposed approach. The implementation of the proposed architecture is available in a GitHub repository [9].

IV. RESULTS AND ANALYSIS

The proposed CGRA interconnect architecture was synthesized and implemented on a Kria KV260 Vision AI Starter Kit FPGA using Xilinx Vivado.

The critical path delay was estimated using the relationship between clock period and worst negative slack, as defined in (1):

$$T_{critical} = T_{clock} - WNS \quad (1)$$

The maximum operating frequency can then be estimated from the critical path delay as defined in (2):

$$F_{max} = \frac{1}{T_{critical}} \quad (2)$$

To determine the operating frequency of the design, multiple clock constraints were evaluated. This analysis was performed using the 3×3 CGRA configuration, which represents the more demanding case in terms of interconnect complexity. An initial test at 300 MHz resulted in a large negative slack (-9.682 ns), indicating that the design exceeds its timing capabilities.

At 50 MHz, the design meets timing with a large margin, while at 80 MHz, it still meets timing but operates very close to the limit (0.311 ns slack). At 75 MHz, a better balance between performance and timing margin is achieved (WNS = 1.182 ns), making it a more robust operating point.

Based on the critical path delay, the estimated maximum operating frequency is approximately 82 MHz for the 3×3 configuration. Therefore, a practical operating range of 70–75 MHz is recommended to ensure reliable operation. This operating range is expected to be valid for smaller configurations, such as the 2×2 mesh, which exhibit lower routing complexity. Timing results for different target frequencies are presented in Table I.

Table I. Timing analysis (3x3 mesh design)

Frequency (MHz)	Period (ns)	WNS (ns)	Critical Path (ns)	Timing Cond.
50	20	6.7	13.3	Met
75	13.33	1.182	12.15	Met
80	12.5	0.311	12.19	Met
300	3.333	-9.682	13.015	Not met

The design evaluation was performed on 2×2, 3×3, and 4×4 CGRA meshes, analyzing the resource utilization results in Table II to assess the scalability of the interconnect. While the maximum operating frequency was derived from the 3×3 configuration as a representative case, Table II provides an overview of the achievable maximum frequencies across different mesh sizes.

Table II. Interconnect Resource utilization for different CGRA sizes

CGRA Size	LUTs	FFs	F7 Muxes	Global Clk Buffers	Fmax Op. (MHz)
2x2	3988	4882	4	1	300
3x3	7040	7436	68	2	75
4x4	11304	9773	36	2	40

LUT and FF counts grow significantly as the CGRA size increases, due to the higher number of processing elements and, more importantly, the expanded interconnect required between them.

A particularly notable increase is observed in the number of F7 multiplexers. This growth is primarily driven by the expansion of the crossbar-based switching fabric, where each additional node introduces new routing options and connectivity requirements, resulting in a larger number of multiplexing structures.

In terms of performance, the maximum operating frequency decreases as the mesh size increases. This behavior indicates that the critical path becomes dominated by the interconnect, as data must traverse a greater number of switches and longer combinational paths.

Functional validation of the interconnect was performed using a custom testbench covering representative communication patterns across the CGRA grid.

One-to-one communication scenarios were evaluated to validate correct data propagation across multi-hop paths. These tests confirm that data traverses the grid following the statically configured Manhattan routes, reaching the destination node without loss or corruption. Figure 2 illustrates this routing behavior.

Scenarios involving intermediate PEs were also evaluated, where selected nodes perform computations on in-flight data. The results confirm that PEs can operate transparently within the routing path, correctly consuming input data and reinjecting results into the interconnect. As can be illustrated in Figure 4:

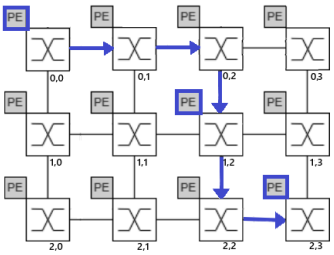

Figure 4. Crossbar flow based on a one-to-one route with middle observation.

One-to-many communication patterns were validated through broadcast scenarios, demonstrating that a single source node can distribute data to multiple destinations simultaneously. These tests confirm the ability of the crossbar-based switches to support concurrent routing paths without conflicts. Figure 5 illustrates this routing behavior.

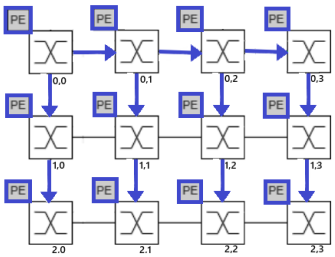

Figure 5. Crossbar flow-based full broadcast route.

Additionally, concurrent multi-route behavior was validated by configuring multiple independent communication paths across the CGRA grid. In these tests, several horizontal routes were activated simultaneously, each injecting distinct data streams from different source nodes.

Two scenarios were evaluated: routing with intermediate processing, where selected PEs apply a NOT operation to in-flight data, and manual routing, where data is forwarded without modification. The results confirm that the interconnect supports concurrent data transfers across multiple paths without interference, while enabling selective computation within the routing fabric. This behavior can be illustrated by Figure 6.

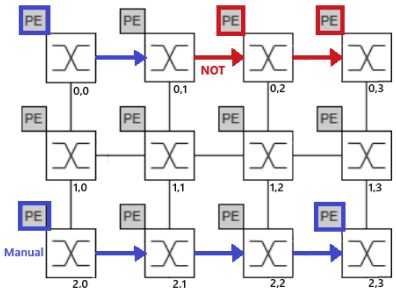

Figure 6. Crossbar flow-based parallel routes.

V. FUTURE WORK

While the proposed architecture demonstrates the feasibility and effectiveness of the current design choices, several opportunities remain for further improvement and exploration. Future work will focus on enhancing performance, scalability, and flexibility by revisiting key architectural decisions and investigating alternative design approaches.

In this initial implementation, combinational switches were adopted to simplify the communication mechanism, as they require fewer control signals and reduce design complexity. However, synchronous switch-based designs present several advantages that merit further exploration. In particular, synchronous

switches can achieve improved timing closure at higher clock frequencies by introducing pipeline registers that break long combinational paths. This also enhances scalability and robustness, especially in larger network topologies.

These benefits come at the cost of increased latency, typically adding one or more clock cycles per stage. Therefore, future work will involve evaluating the trade-offs between combinational and synchronous switching approaches across different system configurations. While combinational switches may remain suitable for smaller systems operating at lower frequencies, synchronous designs are expected to be more appropriate for larger-scale architectures targeting higher performance.

Another promising direction for future work is the exploration of data packetization within the proposed architecture. Although the current design employs a circuit-switching approach for data routing, this does not preclude the possibility of transmitting data sequentially to effectively reuse the existing link bandwidth. By introducing packetization, larger data transfers could be supported over fixed-width links without increasing physical interconnect complexity. This approach would preserve the simplicity of routing and flow control inherent to circuit switching, while enabling more efficient hardware utilization and improved flexibility in handling varying data sizes.

VI. CONCLUSIONS

A circuit-switched interconnect architecture for A circuit-switched interconnect architecture for CGRAs was proposed, reducing routing complexity through static configuration. The elimination of dynamic arbitration and buffering simplifies the interconnect microarchitecture and reduces area consumption, while preserving deterministic communication.

FPGA results show stable timing behavior, achieving up to 80 MHz for the 3×3 mesh configuration, which was used as a representative case for timing characterization. A practical operating range of 70–75 MHz is identified to ensure reliable operation for this configuration and smaller CGRA sizes.

Evaluation across multiple mesh sizes shows that hardware cost increases with the CGRA dimensions, primarily due to the expansion of the interconnect and crossbar-based switching fabric, which introduces additional routing resources and multiplexing structures. At the same time, the maximum operating frequency decreases as the mesh size increases, indicating that the interconnect becomes the dominant factor in the critical path.

Functional validation confirms reliable operation across multiple communication patterns, including one-to-one, one-to-many, and concurrent multi-route scenarios, without interference between data paths. The architecture also supports in-path computation, allowing intermediate PEs to process data while preserving end-to-end communication.

Additionally, the proposed routing model enables predictable data movement through statically defined Manhattan paths and supports parallel data flows under a structured configuration scheme. These results demonstrate that reduced routing flexibility can still sustain representative CGRA communication patterns while simplifying the overall interconnect design.

VII. USE OF AI-ASSISTED TOOLS

In this project, AI tools, including ChatGPT, Copilot, and Grok, were used solely as supportive resources to assist with concept exploration, clarification of ideas, and improvement of the manuscript's wording. All technical decisions, interpretations, and final content were determined by the authors.

VIII. REFERENCES

[1] J. Anderson, B. Adhi, C. Cortes, E. Del Sozzo, O. Ragheb, y K. Sano, "Exploration of Compute vs. Interconnect Tradeoffs in CGRAs for HPC," en Proc. 13th Int. Symp. on Highly Efficient Accelerators and Reconfigurable Technologies (HEART '23), New York, NY, USA: ACM, 2023, pp. 59–68. doi: 10.1145/3597031.3597055.

[2] S. A. Chin, N. Sakamoto, A. Rui, J. Zhao, J. H. Kim, Y. Hara-Azumi, y J. Anderson, "CGRA-ME: A unified framework for CGRA modelling and exploration," en IEEE ASAP, 2017, pp. 184–189.

[3] Sichao Chen, Chang Cai, Su Zheng, Jiangnan Li, Guowei Zhu, Jingyuan Li, Yazhou Yan, Yuan Dai, Wenbo Yin, and Lingli Wang. 2024. HierCGRA: A Novel Framework for Large-scale CGRA with Hierarchical Modeling and Automated Design

Space Exploration. ACM Trans. Reconfigurable Technol. Syst. 17, 2, Article 35 (June 2024), 31 pages. https://doi.org/10.1145/3656176

[4] Vieira, M., Canesche, M., Bragança, L., Campos, J., Silva, M., Ferreira, R., & Nacif, J. A. (2021, May). Reshape: A run-time dataflow hardware-based mapping for cgra overlays. In 2021 IEEE International Symposium on Circuits and Systems (ISCAS) (pp. 1-5). IEEE.

[5] K. Kutsuna, T. Kojima, H. Takase, and H. Nakamura, "An Area-Efficient Coarse-Grained Reconfigurable Array Design for Approximate Computing," 2023 IEEE 16th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoC), pp. 59–64, Dec. 2023, doi: 10.1109/MCSoC60832.2023.00017

[6] T. Patil and A. Sandi, "Design and implementation of asynchronous NOC architecture with buffer-less router," Materials Today: Proceedings, vol. 49, pp. 756–763, 2022, doi: https://doi.org/10.1016/j.matpr.2021.05.282.

[7] Z. Li et al., "Enhancing CGRA Efficiency Through Aligned Compute and Communication Provisioning," Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1, pp. 410–425, Mar. 2025, doi: https://doi.org/10.1145/3669940.3707230.

[8] Karunaratne, M., Mohite, A. K., Mitra, T., & Peh, L. S. (2017, June). HyCUBE: A CGRA with reconfigurable single-cycle multi-hop interconnect. In Proceedings of the 54th Annual Design Automation Conference 2017 (pp. 1-6).

[9] A. Cordoba, J. Carranza, J. Ureña and V. Sanchez. "Cgra-Circuit-Cwitch," GitHub Repository, 2026. [Online]. Available at: https://github.com/acordobav/cgra-circuit-switch.git